JAMIN PROPERTIES — MOBILE APP

Claude Code Super Prompt · "Signature for Fortune"

> **Build target:** A production-grade, cross-platform **mobile application** (iOS + Android) for the Jamin Properties real-estate sales ecosystem. Same 16-module architecture as the platform blueprint, rebuilt mobile-native, plus a **Dynamic Digital Business Card** module.
> **Prime directive:** *Everything is dynamic. Nothing is hardcoded.* Roles, hierarchies, projects, plans, property types, commission models, forms, brochure/card templates and referral rules are all data — configurable by the Super Admin at runtime. The app reads configuration; it never bakes it in.

---

0. HOW TO USE THIS PROMPT

You (Claude Code) will build this app in the **phased sequence** in §15. After each phase: typecheck, lint, run, and produce a short "what I built / what's next" note. Do **not** skip ahead. Ask before introducing any dependency not listed in §2. Treat §13 (Dynamic Rule) and §14 (Money & Compliance) as non-negotiable guardrails on every file you touch.

Open decisions flagged for confirmation (proceed with the stated default unless told otherwise):

- **D1 — Framework:** default **Expo (React Native) + Expo Router + TypeScript**. (Flutter is the alternative; if chosen, port the same architecture.)
- **D2 — Phone verification:** phone OTP is **optional** at onboarding (email OTP is mandatory). Default: collect phone, verify lazily before first payout/withdrawal.
- **D3 — Wallet payouts:** default to a **ledger-first** model (record payouts; integrate a real PSP/UPI rail behind an adapter interface later).

---

1. BRAND & DESIGN LANGUAGE

Pulled from the official logo. Use these exact tokens.

```ts
```

```ts
// theme/tokens.ts
export const color = {
  red:        '#FD0001', // primary brand
  redDeep:    '#C70000', // gradients, pressed states
  gold:       '#FBBC15', // accent, markers, CTAs-secondary
  goldDeep:   '#C8911E', // fine rules, "signature" text
  ink:        '#1A1A1A', // primary text
  charcoal:   '#202020', // headings, dark surfaces
  muted:      '#74746E', // secondary text
  line:       '#E8E7E2', // hairlines, borders
  paper:      '#F7F7F5', // app background (light)
  surface:    '#FFFFFF',
  success:    '#1E9E5A', danger: '#D4351C', warn: '#E6A100',
};
export const tagline = 'Signature for Fortune';
```

Typography

- Display/Headings & body: **TT Norms Pro** (licensed — place files in `assets/fonts/`). **Fallback: Inter** (bundle via `@expo-google-fonts/inter`). Wire the family so TT Norms is preferred and Inter is the graceful fallback.
- Numbers / codes / money / plot codes: **JetBrains Mono** (tabular, tight tracking).
- Weights in use: 400 / 500 / 600 / 700 / 800.

Visual signature

- Light "luxury-institutional" surface: paper background, white cards, hairline borders, generous spacing.
- **Red→deep-red gradient** for primary blocks/badges; **gold** for accents, the small rotated-diamond list marker, and the section corner-cut.
- Section/number badges: rounded square, red gradient, a small gold triangular corner-cut (mirrors the logo's gold corner).
- Money in JetBrains Mono, INR `₹` with Indian digit grouping (e.g. `₹1,25,00,000`).
- Motion: restrained. 150–220ms ease; spring on card flip and share sheet only.
- Dark mode: invert paper→`#121212`, surface→`#1C1C1C`, keep red/gold accents.

Accessibility: WCAG AA contrast, dynamic type support, hit targets ≥44pt, screen-reader labels on every interactive element.

---

2. TECH STACK (locked)

Layer	Choice
App	Expo SDK (latest stable) + **Expo Router** (file-based nav) + TypeScript (strict)
Styling	**NativeWind** (Tailwind for RN) mapped to the tokens above + a small primitive component lib
Data/State	**TanStack Query** (server state) + **Zustand** (local/session/UI state)
Backend	**Supabase —** Postgres, Auth (email OTP), Storage, Realtime, **Edge Functions** (Deno)
Money math	**decimal.js —** MANDATORY for all monetary/commission math. **No IEEE-754 floats for money, ever.**
Forms	**react-hook-form** + **zod** (schemas generated from dynamic form definitions)
Camera/Geo	`expo-camera`, `expo-location`, `expo-media-library`
QR	`react-native-qrcode-svg` (render) + a scanner via `expo-camera`
Card/Ad render	`react-native-view-shot` (rasterise card & ad creatives to high-res PNG)
Sharing	`expo-sharing` + RN `Share` + channel deep-links; vCard `.vcf` export
Notifications	`expo-notifications` (push) + Supabase Realtime (in-app)
AI	**Anthropic Claude API** called **only** from a Supabase Edge Function (key stays server-side)
Secure store	`expo-secure-store` (tokens), `expo-local-authentication` (biometric unlock)
i18n	`i18next` (English default; structure for Indic languages — Hindi, Tamil, Telugu, Kannada, Marathi, Bengali, Gujarati)
Testing	Jest + React Native Testing Library; Detox (smoke E2E)

Do not add libraries beyond this without asking.

---

3. MONOREPO / PROJECT STRUCTURE

```
jamin-mobile/
├── app/                        # Expo Router routes
│   ├── (auth)/                 # install → email → otp → kyc → profile
│   ├── (onboarding)/
│   ├── (tabs)/                 # role-aware bottom tabs
│   │   ├── home/               # dashboard (role-scoped)
│   │   ├── properties/         # buyer discovery
│   │   ├── card/               # ★ Digital Business Card
│   │   ├── network/            # hierarchy / team / referrals
│   │   └── wallet/             # commissions, earnings, withdrawals
│   ├── property/[code].tsx
│   ├── brochure/…  ad-creator/…  forms/[id].tsx  admin/…
│   └── _layout.tsx
├── src/
│   ├── components/             # primitives + branded components
│   ├── features/               # one folder per module (§5)
│   │   ├── hierarchy/ inventory/ buyer/ agent/ promoter/
│   │   ├── brochure/ referral/ commission/ wallet/
│   │   ├── card/ ★  photo-ad/ forms/ ai/ gamification/ admin/
│   ├── lib/                    # supabase client, money(decimal), qr, share,
vcard, geo
│   ├── theme/                  # tokens, fonts, nativewind preset
│   ├── stores/                 # zustand
│   ├── api/                    # query hooks per domain
│   └── types/                  # generated supabase types + domain types
├── supabase/
│   ├── migrations/             # SQL (every table + RLS)
│   ├── functions/              # edge functions (ai, payouts, card-track,
referral-attribute)
│   └── seed/                   # demo config (roles, sample
project/plan/types)
├── assets/fonts/               # TT Norms (licensed) + Inter + JetBrains Mono
└── docs/                       # CLAUDE.md, ARCHITECTURE.md, RLS.md
```

Create a `CLAUDE.md` governance file at root capturing the dynamic rule, money rule, brand tokens, and "ask before new deps."

---

4. ONBOARDING & AUTH FLOW (build first, exactly this order)

> The user is asked to **install the app**, then logs in with **email + OTP**. First-time users complete profile + KYC and are auto-placed in the hierarchy.

1. **Splash / brand intro —** logo, tagline, 1–2 value lines. Detect deep-link/referral params here and stash them (see §8).
2. **Email entry →** `supabase.auth.signInWithOtp({ email })`. Validate, rate-limit, friendly errors.
3. **OTP screen —** 6-digit, auto-advance inputs, resend timer, paste support, autofill from SMS/email where possible. Verify → session.
4. **New vs returning:**
   - Returning + profile complete → straight to **Home**.
   - New → onboarding stepper.
5. **Profile creation —** full name, profile photo (camera/library, cropped square), phone (D2: collect now, verify later), preferred language, optional designation hint.
6. **KYC capture —** document type + upload (front/back), live selfie, consent checkboxes (DPDP). Status: `pending → verified/rejected`. App is usable in a limited mode while `pending`; gated actions (payouts, listing) require `verified`.
7. **Referral binding —** if a referral code / card / brochure / QR brought the user in, resolve the parent and **bind** `parent_id`; otherwise assign per Super-Admin referral rules (round-robin / territory / default sponsor).
8. **Auto hierarchy assignment —** set role (default lowest applicable, e.g. Buyer/Agent per invite context), compute hierarchy path, generate `referral_code`, `referral_link`, **referral QR**, and provision the user's **Digital Business Card** (§6).
9. **Welcome → role-aware Home.** Offer biometric unlock opt-in.

Auth guardrails: secure-store the session, refresh handling, biometric re-auth on resume (configurable), full sign-out wipe.

---

5. THE 16 MODULES → MOBILE (feature spec)

Build each as a `src/features/<name>` slice with screens, query hooks, and config-driven UI. Everything below is **read from DB config**, never hardcoded.

01 · User Hierarchy & Commission Network — Unlimited multi-level tree (Super Admin → State Head → Regional Manager → Promoter → Sub Promoter → Agent → Buyer, *and beyond* — depth is unlimited). Role-based permissions, team management, territory assignment, recruitment, performance + commission tracking, referral-network map (interactive,

zoomable tree/graph). Use `parent_id` + a materialised path (Postgres `ltree` or closure table) for unlimited depth and fast subtree queries.

02 · Registration & Onboarding — as §4. Email/mobile registration, OTP, KYC, profile, referral-ID + QR + link generation, automatic hierarchy assignment.

03 · Inventory Management — Dynamic **Projects → Plans → Property Types → Plots**, all unlimited. **Dynamic plot coding** (pattern-based, e.g. `LD-0001`, `VL-0001`, `AP-0001`; pattern defined in config). **Auto-Replacement Engine:** on `plot.sold` → mark inventory sold → trigger commissions → update availability → promote next available plot. Implement as a Postgres trigger + Edge Function so it's atomic and audited.

04 · Buyer App — Property search, advanced filters, **map search** (`react-native-maps`/Expo Maps) with nearby amenities, property details, photo gallery, video tours, virtual tours, **3D walkthroughs** (WebView/embedded), brochure downloads, wishlist, compare, **EMI calculator**, **ROI calculator** (Decimal.js), enquiry forms (dynamic), site-visit booking, live chat, booking engine, payment tracking.

05 · Agent Portal — Lead management/assignment/tracking, follow-up scheduler (with reminders/push), property sharing, marketing-material access, referral links, commission dashboard, earnings reports, **wallet**, withdrawals, team building, performance metrics.

06 · Promoter Portal — Recruit agents/promoters, team hierarchy + monitoring, team revenue/sales, commission reports, **bonus structures**, incentive tracking, territory management, performance dashboards.

07 · Smart Brochure System — Brochure library (project brochures, flyers, images, videos, offers, campaigns). **Automatic personalization**: user name, phone, photo, referral code, QR, company branding, tracking info — bound to template fields. **Sharing channels**: WhatsApp, Telegram, **KaroChat**, Facebook, Instagram, LinkedIn, X, SMS, Email, Direct Link. Render personalised brochure to image/PDF on-device before share.

08 · Viral Referral Engine — Flow: *Share brochure/card → prospect clicks link → prospect registers → OTP → auto-assign to hierarchy.* Tracking: referral, QR, campaign, cookie/identifier, device fingerprint. **Fraud detection** (velocity, device reuse, self-referral, geo anomalies). Every shareable artifact carries a unique tracking token that attributes downstream registrations to the sharer. (The Business Card is a first-class entry point here.)

09 · Property Photo Ad Creator — Take/upload picture, **GPS capture**, timestamp, location verification. **Auto-generate advertisement** stamped with: user name, phone, QR, referral link, "Real Property / Captured Live / Taken On Site", date, time, location, contact info. **Export formats**: WhatsApp Status, Story, Social Post, Flyer, Banner, Marketing Creative — render via view-shot at correct aspect ratios. Watermark + branding locked by template.

10 · Dynamic Commission Engine — Commission by property/project/plan, **team override**, performance bonuses, incentive programs, slab-based rewards, revenue sharing.

Commission wallet, automated payouts, **immutable audit trail**. Rules live in `commission_rules` (config). Engine = pure, deterministic, Decimal.js, fully testable; emits ledger entries only (never mutates balances directly).

11 · Dynamic Form Builder — Buyer/agent/promoter/property/KYC/lead/booking forms + custom fields, **drag-&-drop builder**, unlimited fields. Forms are `form_definitions` + `form_fields`; the app renders + validates them at runtime (zod schema derived from definition). Admin builds; app consumes.

12 · Admin Portal (mobile-optimised, Super Admin / privileged roles) — Property, builder, user, inventory management; commission rules; marketing assets; KYC management; approval workflows; payment approvals; CRM; reporting; analytics; audit logs; **system configuration** (this is where "dynamic everything" is edited).

13 · Backend Services — Auth, User, Property, Inventory, CRM, Referral, Commission, Wallet, Notification, Reporting, Analytics, AI, Media-Processing — implemented as Supabase tables + RPC + Edge Functions with clean domain boundaries.

14 · AI Features — AI property descriptions, flyer creation, social posts, marketing campaigns, **lead scoring**, sales assistant, photo enhancement, brochure generation, video scripts. All via **Claude API behind an Edge Function**. Stream where useful. Cache + rate-limit. Never expose the key client-side.

15 · Gamification — Leaderboards, achievement badges, top agent/promoter awards, sales & referral milestones, team-growth awards, bonus rewards. Realtime leaderboard via Supabase Realtime; badges as config-driven rules.

16 · Core Platform Rule — The governing constraint (§13). Surface a read-only "Platform Principles" screen in Admin reflecting it.

---

6. ★ DIGITAL BUSINESS CARD (new module — full spec)

Goal: Every user gets a branded **digital business card** that is **auto-filled dynamically from their profile** and is **shareable** to anyone. It doubles as a **referral entry point** — when a recipient scans/taps it and registers, the card owner gets the attribution.

6.1 Auto-filled, always-live fields

Bound to the user's profile and hierarchy — edits to profile reflect on the card instantly (no manual re-creation):

- Profile photo (or initials avatar), **Full name**
- **Designation** — derived from the user's role in the hierarchy (e.g. "Regional Manager", "Agent"); admin can map role → display title

- **Phone**, **Email**, **WhatsApp**
- **Company**: "Jamin Properties" + logo + tagline "Signature for Fortune"
- **Referral code** + **referral QR** (encodes the tracked referral/card link)
- Optional: territory/branch, social links, website, short bio/specialisation, languages spoken
- Optional: RERA / license number field (config-driven; only shown if present)

6.2 Templates (dynamic — no hardcoding)

- `card_templates` table defines layouts (front + optional back), field bindings, color/style variants, QR style, which optional fields show. Brand-locked elements (logo, tagline, palette) cannot be overridden by users; admin controls templates.
- User can pick from **admin-approved templates** and choose accent variant (within brand).
- Live preview with **flip animation** (front ⇄ back).

6.3 Generation & rendering

- Provisioned automatically at end of onboarding (§4.8); regenerates the rendered asset whenever bound fields change.
- Render high-res PNG via `react-native-view-shot` for image sharing; keep a vector/HTML render for crisp on-screen display.

6.4 Sharing (every channel + contact save)

- **Native share sheet** (image + link).
- **vCard** `.vcf` export so the recipient saves a real contact (name, phone, email, org, title, photo, URL with referral param). Build a spec-compliant vCard 3.0/4.0 string in `lib/vcard.ts`.
- Direct channels: **WhatsApp, Telegram, KaroChat, Facebook, Instagram, LinkedIn, X, SMS, Email, Direct Link**, AirDrop/Nearby.
- Every shared artifact carries a **unique** `card_share` **token** → links to §8 referral attribution.
- Optional: **NFC tap-to-share** (`expo` NFC where supported); **Apple Wallet / Google Wallet pass** as a stretch goal.

6.5 Scan / deep-link → referral funnel

- Card QR / link → opens app (or store, then app on first run) with `?ref=<code>&card=<card_id>&t=<token>`.

- New registration attributes to the card owner and **binds** `parent_id` per referral rules. (§8)
- Each scan/open is logged in `card_scans` (timestamp, geo if permitted, device, converted?).

6.6 Card analytics (owner-facing screen)

- Views, shares, scans, contact-saves, **registrations converted**, and downstream commission earned via the card.
- Per-channel breakdown; trend over time.

6.7 Data model (additions)

```
business_cards(id, user_id, template_id, accent_variant, optional_fields
jsonb,
              render_url, vcard_cache, status, created_at, updated_at)
card_templates(id, name, layout jsonb, field_bindings jsonb, allowed_variants
jsonb,
              brand_locked jsonb, is_active, created_by)
card_shares(id, card_id, sharer_user_id, channel, token unique, created_at)
card_scans(id, card_id, token, ip, device, geo, registered_user_id nullable,
          converted bool, created_at)
```

RLS: a user reads/writes only their own `business_cards`; `card_templates` readable by all, writable by admin; `card_scans`/`card_shares` insertable by edge function, readable by owner + admin.

---

7. DATA MODEL (core tables — generate full SQL + RLS in `supabase/migrations/`)

Config/dynamic tables (Super-Admin editable): `roles`, `permissions`, `territories`, `projects`, `plans`, `property_types`, `commission_rules`, `form_definitions`, `form_fields`, `card_templates`, `brochure_templates`, `gamification_rules`, `referral_rules`, `system_config`.

Core entities:

```
profiles(id=auth.uid, role_id, parent_id, hierarchy_path ltree, referral_code
unique,
         designation, full_name, photo_url, phone, phone_verified, email,
kyc_status,
         territory_id, language, status, created_at)
properties(id, project_id, plan_id, property_type_id, plot_code unique, price
numeric,
           status[available/reserved/sold], coordinates, media jsonb, attrs
jsonb)
inventory_events(id, property_id, type, actor_id, payload jsonb, created_at)
-- audit + auto-replace
leads(id, owner_id, source, status, contact jsonb, property_id, score,
created_at)
follow_ups(id, lead_id, due_at, note, status)
referral_events(id, sharer_id, artifact_type[card/brochure/ad/link], token,
channel,
                prospect_id nullable, stage, device jsonb, geo, fraud_score,
created_at)
commission_rules(id, scope[property/project/plan/team/bonus/slab], match
jsonb,
                 formula jsonb, currency, priority, active)
commission_ledger(id, user_id, source_ref, rule_id, amount numeric,
direction, status,
                  immutable=true, created_at)            -- append-only
wallets(id, user_id, balance numeric)                    -- derived from
ledger; never hand-edited
withdrawals(id, user_id, amount numeric, status, rail, requested_at,
settled_at)
bookings(id, property_id, buyer_id, agent_id, status, amount, schedule)
payments(id, booking_id, amount numeric, status, method, txn_ref)
brochures(...)  shares(...)  ad_creatives(...)  forms_submissions(...)
badges(...)  achievements(...)  leaderboards(...)  awards(...)
notifications(...)  ai_generations(...)  audit_logs(immutable)
+ business_cards / card_templates / card_shares / card_scans (§6.7)
```

RLS everywhere. Default deny. Policies scope rows by `auth.uid()`, by role, and by **subtree** (a manager sees their downline via `hierarchy_path <@ my_path`). Provide a tested `RLS.md` documenting every policy.

---

8. REFERRAL ATTRIBUTION & DEEP LINKING (cross-cutting)

- Universal links / app links + Expo Router deep links: `jamin://`, `https://jamin.app/r/<token>`.
- Resolution order on first open: explicit `?ref=` / `card` token → stored install-referrer → default rule.
- Edge Function `referral-attribute`: validates token, runs fraud checks (self-referral, device reuse, velocity, geo), sets `parent_id`, writes `referral_events`, fires "new downline" notification + any signup bonus.
- The **business card**, **smart brochure**, and **photo ad** all mint tracked tokens through the same pipeline.

---

9. EDGE FUNCTIONS (Deno)

`ai-generate` (Claude API: descriptions/flyers/posts/scoring/scripts, streaming) · `referral-attribute` · `card-track` (scan/share logging) · `inventory-autoreplace` · `commission-run` (deterministic, Decimal.js, append-only ledger) · `payout-initiate` (adapter behind PSP/UPI) · `kyc-review` (hooks/webhooks) · `notify` (push fan-out). All validate JWT, enforce RBAC, log to `audit_logs`.

---

10. AI INTEGRATION RULES

- Claude API key **only** in Edge Function env. Client calls the function, never Anthropic directly.
- Each AI feature = typed request/response contract; stream long outputs; cache by input hash; per-user rate limit; log to `ai_generations`.
- Lead scoring returns a 0–100 score + rationale; store on `leads.score`.

---

11. NOTIFICATIONS

Push (expo-notifications) + in-app (Realtime): new lead, lead assigned, follow-up due, new downline/referral converted, commission credited, withdrawal status, KYC status, milestone/badge unlocked, booking updates. Preferences screen (per-type toggles, quiet hours).

---

12. PERFORMANCE, OFFLINE, SECURITY

- TanStack Query caching + optimistic updates; skeleton loaders; image caching/CDN; lazy routes.
- Offline-friendly reads (cache last data); queue mutations where safe.
- Secure store for tokens; biometric unlock; certificate pinning (where feasible); jailbreak/root soft-warning.
- Crash + analytics (Sentry — already in the ecosystem; gate PII).
- All money via `lib/money.ts` (Decimal.js wrapper) — lint rule/CI check to forbid raw `+ - * /` on currency values.

---

13. THE DYNAMIC RULE (hard guardrail — applies to every screen)

Nothing in this list may be hardcoded; all are read from config tables and editable by Super Admin at runtime:

- ❌ No hardcoded **forms · commissions · projects · property types · hierarchies · referral rules** · roles · plot-code patterns · card/brochure templates · gamification rules.
- ✅ Unlimited **projects · plots · plans · users · hierarchy levels · commission models.**
- ✅ **Fully configurable by Super Admin.**

If you ever find yourself writing an `enum`, a fixed array of roles/types, or a literal commission %, stop — move it to config.

---

14. MONEY & COMPLIANCE

- **Decimal.js mandatory** for every monetary/commission/EMI/ROI calculation. No floats. No rounding drift — define rounding mode centrally.
- INR formatting with Indian grouping + `₹`; JetBrains Mono for all figures.
- Immutable `audit_logs` and append-only `commission_ledger` (wallet balances derived, never directly edited).
- DPDP-aligned consent capture at KYC; data-export/delete hooks. Keep KYC media in a restricted Storage bucket with signed-URL access only.

---

15. BUILD SEQUENCE (execute in order; stop & report after each phase)

- **P0 — Foundation:** Expo app, Expo Router, NativeWind + tokens, fonts (TT Norms→Inter, JetBrains Mono), Supabase client, CLAUDE.md, base primitives, navigation shell.
- **P1 — Auth & Onboarding (§4):** email OTP, profile, KYC, referral binding, hierarchy assignment, referral code/QR, **provision business card stub**.
- **P2 — Dynamic config & inventory (§5.01–03):** roles/permissions, projects/plans/types, properties + dynamic plot codes, auto-replacement trigger, hierarchy tree + RLS subtree policies.
- **P3 — Buyer App (§5.04):** search/filters/map, details, tours, calculators (Decimal.js), enquiry (dynamic form), booking + payment tracking.
- **P4 — Partner portals (§5.05–06):** leads, follow-ups, team building, dashboards, wallet shell.
- **P5 — Commission + Referral engines (§5.08,10 / §8):** `commission_rules`, deterministic engine + ledger, wallet/withdrawals, attribution pipeline + fraud checks.
- **P6 — ★ Business Card (§6) + Smart Brochure (§5.07) + Photo Ad Creator (§5.09):** templates, dynamic fill, view-shot render, vCard, all share channels, scan tracking + analytics, camera/GPS ad creator.
- **P7 — Dynamic Form Builder + Admin Portal (§5.11–12):** runtime form render/validation, admin config surfaces (this unlocks "dynamic everything").
- **P8 — AI Features (§5.14 / §9–10):** Claude Edge Function + the 9 AI features.
- **P9 — Gamification + Notifications + Analytics + Audit (§5.15 / §11–12).**
- **P10 — Hardening & store prep:** full RLS pass + RLS.md, tests, i18n scaffolding, biometric, app icons/splash (brand), EAS build config, store metadata.

---

16. DEFINITION OF DONE (per phase + overall)

- TypeScript strict passes; lint clean; no `any` in domain code.
- Every new table has **RLS** + a documented policy; default-deny verified.
- No hardcoded config (grep for enums/literals in the §13 list returns nothing).
- All money paths use `lib/money.ts`; CI check passes.
- Business card: auto-fills from profile, updates live on profile edit, renders to image, exports vCard, shares on all channels, and a scan that converts attributes to the owner end-to-end (prove with one test).
- Onboarding works cold: install → email → OTP → profile → KYC → home, with referral attribution when launched from a shared card.

- A short demo seed (`supabase/seed/`) lets a fresh dev see the whole flow.

---

17. FIRST REPLY EXPECTED FROM YOU (Claude Code)

1. Confirm D1–D3 (or accept defaults).
2. Scaffold **P0** and show the tree + the rendered branded splash/login.
3. List exact dependency versions you'll install (from §2 only) for approval.
4. Then proceed to P1.

> Brand reminder on every screen: red `#FD0001`, gold `#FBBC15`, ink `#202020`, paper `#F7F7F5`, JetBrains Mono for numbers, and the line **"Signature for Fortune."**